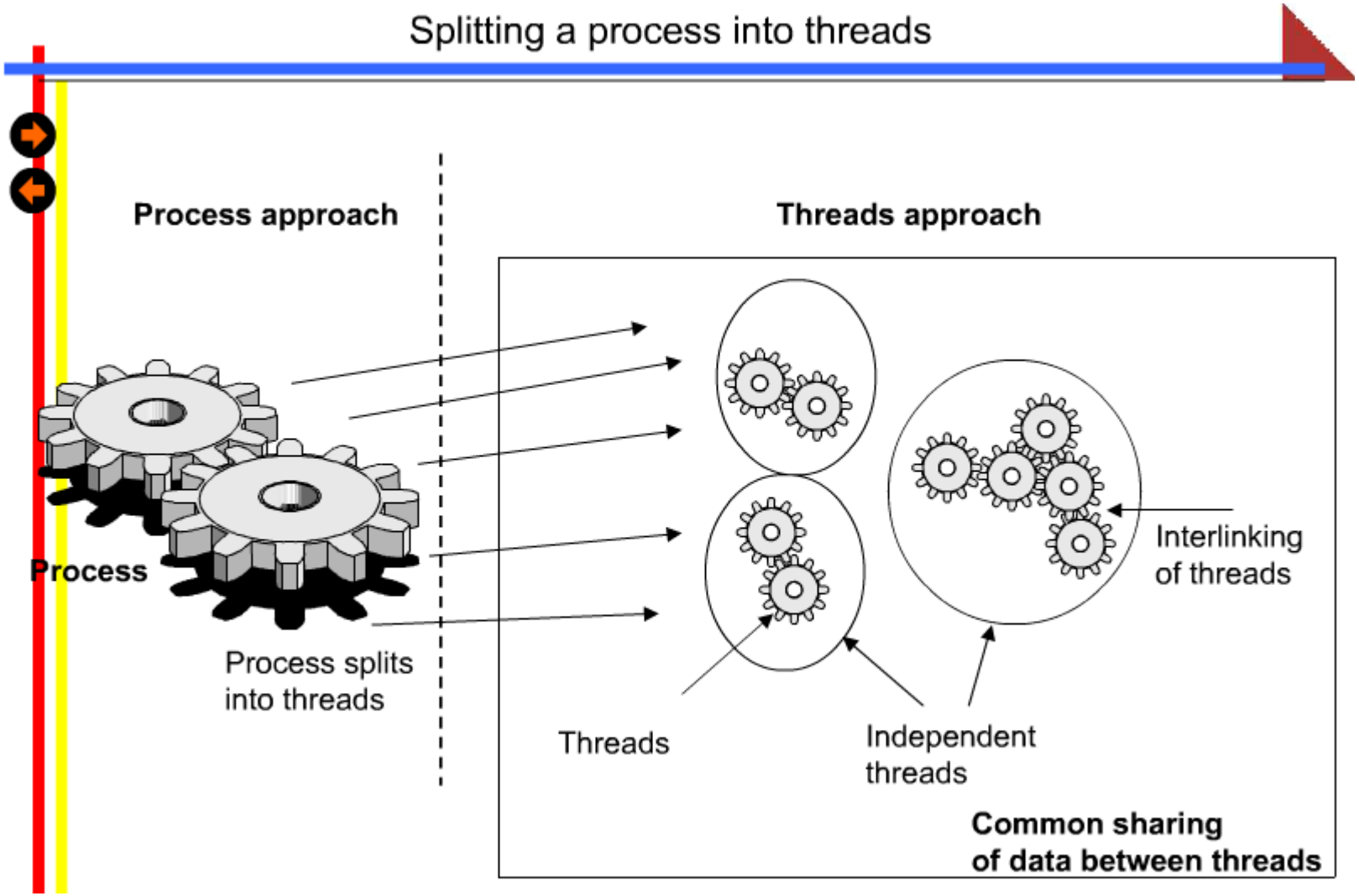
İşletim Sistemleri
(Operating Systems)



Thread (İşlemcik)

Thread (İşlemcik) Nedir?

- Bir process tekil bir thread kontrolü ile yürütülen bir yapıdır (heavy-weight process).

- Bir process’in, aynı adres uzayını kullanarak işlemlerini gerçekleştiren "hafif ağırlıklı process’lere" ise (light-weight process) " **thread**" adı verilir.

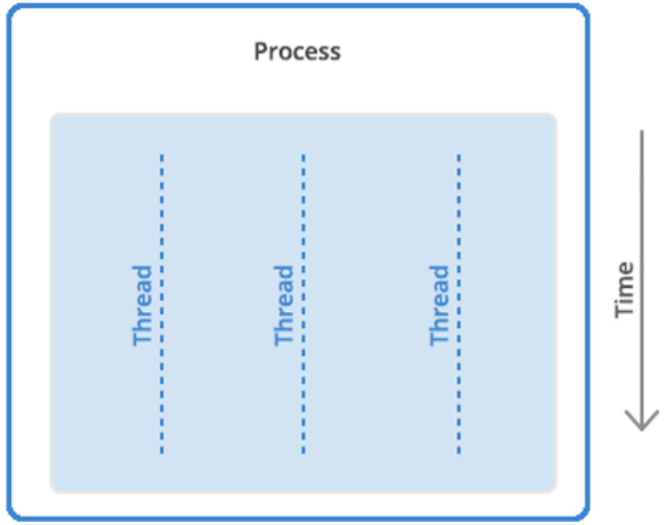
- Process bir kol olarak düşünülürse, thread’lar parmaklar olarak tanımlanabilir.

Thread (İşlemcik) Nedir?

- **Thread kullanımının amaçları:**
- Birden fazla işlemci veya işlemci çekirdeğinden faydalanarak gerçek paralel yürütme olanağı elde etmek,
- Karmaşık operasyonları daha yönetilebilir daha küçük yürütme birimlerine bölmek,
- Kodlamada modülerliği teşvik etmek,
- Daha az kaynak kullanılarak kaynak kullanımını, cevap verme yeteneğini ve verimliliği aynı anda artırmak,

Thread (İşlemcik) Nedir?

- Thread'ler process'lerin içinde yer alır, onları oluştururlar. Yani thread'lerin varlıkları bir process'e bağlıdır.
- Bir process tek bir thread veya birden fazla thread'e sahip olabilir.
- Process ile thread arasındaki birincil fark, thread'lar içinde bulundukları process'in bellek adres uzayını kullanırlar, process'ler ise ayrı ayrı bellek alanlarını kullanırlar.

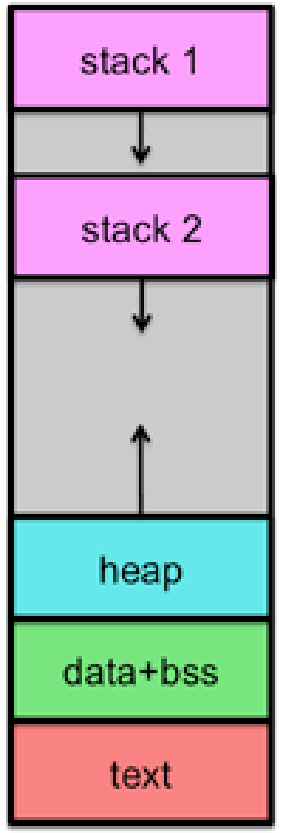


Thread (İşlemcik) Nedir?

- Her thread aynı process'e ait diğer thread'ler ile process'in text/code ve data kısmını ve diğer işletim sistemi kaynaklarını (açık dosyalar, signals) ortak kullanır.

- Her thread kendine ait thread ID, program counter (PC), register set ve stack içerir.

- Thread'ların kendine ait stack ve register'leri thread'ler arasında paylaşılamaz.



İki thread'a sahip bir process
İçin bellek modeli

Multi-processing ve Multi-threading kavramları

- **Multi-processing**, işletim sisteminin aynı anda birden fazla process eşzamanlı/görünürde paralel (concurrency) veya gerçek paralel (parallel) yürütebilmesidir,
- Multi-processing için illaki çok işlemci gereksinimi yoktur.
- **Eşzamanlı/görünürde paralel yürütme:** Aslında tek işlemcide aynı anda sadece tek bir process yürütülür. Birden fazla process mantıksal olarak tek bir işlemcide aynı anda yürütülüyor gibi algılanırken, Process'ler işlemci zamanını paylaşır ve bir process'in işlemcide yürütülme süresi bittiği yerde diğer process yürütülmeye başlanır.
- **Gerçek paralel yürütme:** Çok çekirdekli bir işlemcide birden fazla process fiziksel olarak ayrı ayrı işlemci çekirdeklerinde aynı anda yürütülür.

Multi-processing ve Multi-threading kavramları

Multi-threading, bir process'in adres alanında birden çok thread'in eşzamanlı/görünürde paralel veya gerçek paralel çalışmasıdır.

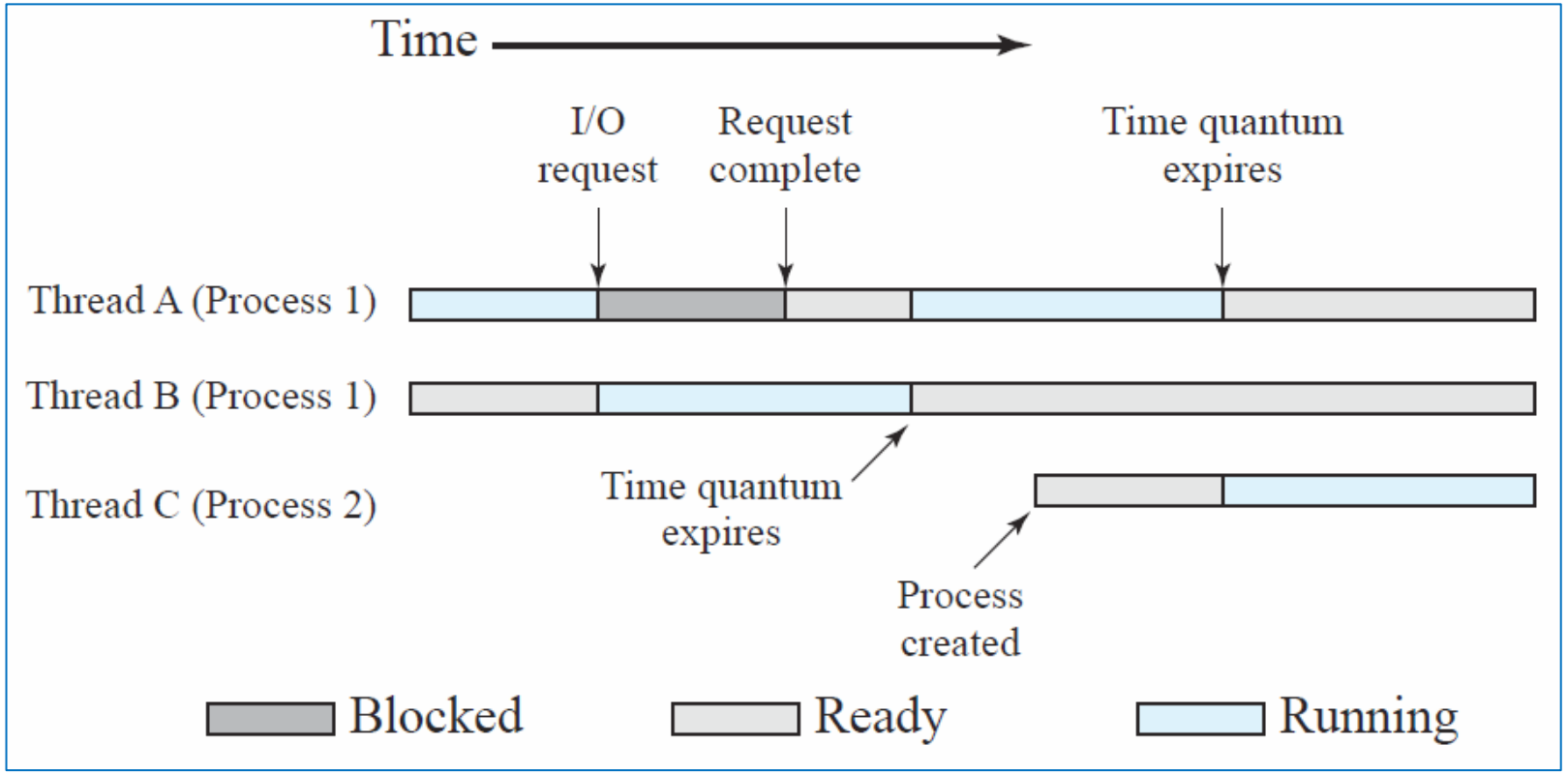
Tek işlemcili sistemlerde, bir process'e adanan sürede, process'in sahip olduğu thread'ler zaman paylaşımlı olarak çalışırlar dolayısıyla gerçek paralel yürütme sağlanamaz.

Tek işlemci, farklı thread'lar arasında çok hızlı geçiş yapar ve eşzamanlı/görünürde paralel bir yürütme gerçekleştirir.

Çok işlemci veya çok çekirdekli işlemciye sahip sistemlerde thread'lar gerçek paralel olarak yürütülebilir.

Multi-threading

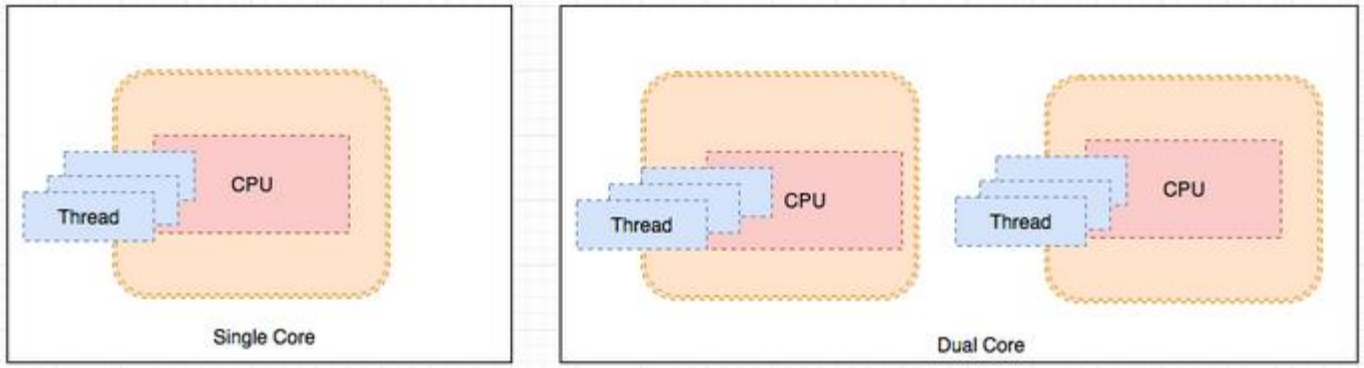
- Tek çekirdekli işlemci üzerinde, birden çok thread **eşzamanlı/görünürde paralel** yürütülebilir.
- Şekil'deki örnekte, iki process'deki üç thread tek bir işlemcide yürütülmektedir.
- İşlemcide yürütülen thread waiting/blocked durumuna geçtiğinde veya yürütülme zamanı bittiğinde işlemciye başka bir thread atanır.



Multithreading on a Uniprocessor

Multi-threading

- Bilgisayar tasarımındaki en önemli gelişmelerden birisi, çok işlemcili/çekirdekli sistemlerin geliştirilmesidir.
- Son zamanlarda, tek chip içerisine birden fazla çekirdek yerleştirilmektedir ve bu tür sistemler multi-core olarak adlandırılır.
- Her bir çekirdek işletim sistemi için ayrı bir işlemci olarak görünür.
- Bir çekirdek üzerinde çalışan çoklu thread'e sahip bir process için eşzamanlı çalışma, thread'lerin belirli zaman aralıklarında yürütülmesini ifade eder **(görünürde paralellik)**.
- Çok işlemci veya çekirdeğe sahip sistemlerde, her çekirdeğe bir thread atanarak thread'lerin paralel yürütülmesi sağlanır **(gerçek paralellik)**.



Multi-threading

- Amdahl kuralına (Gene Amdahl) göre çekirdek sayısına göre bir sistemdeki performans artışı aşağıdaki gibi ifade edilir:
- Burada, S uygulamada eşzamanlı çalışması zorunlu olan kısmın oranını, N ise çekirdek sayısını ifade eder.

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

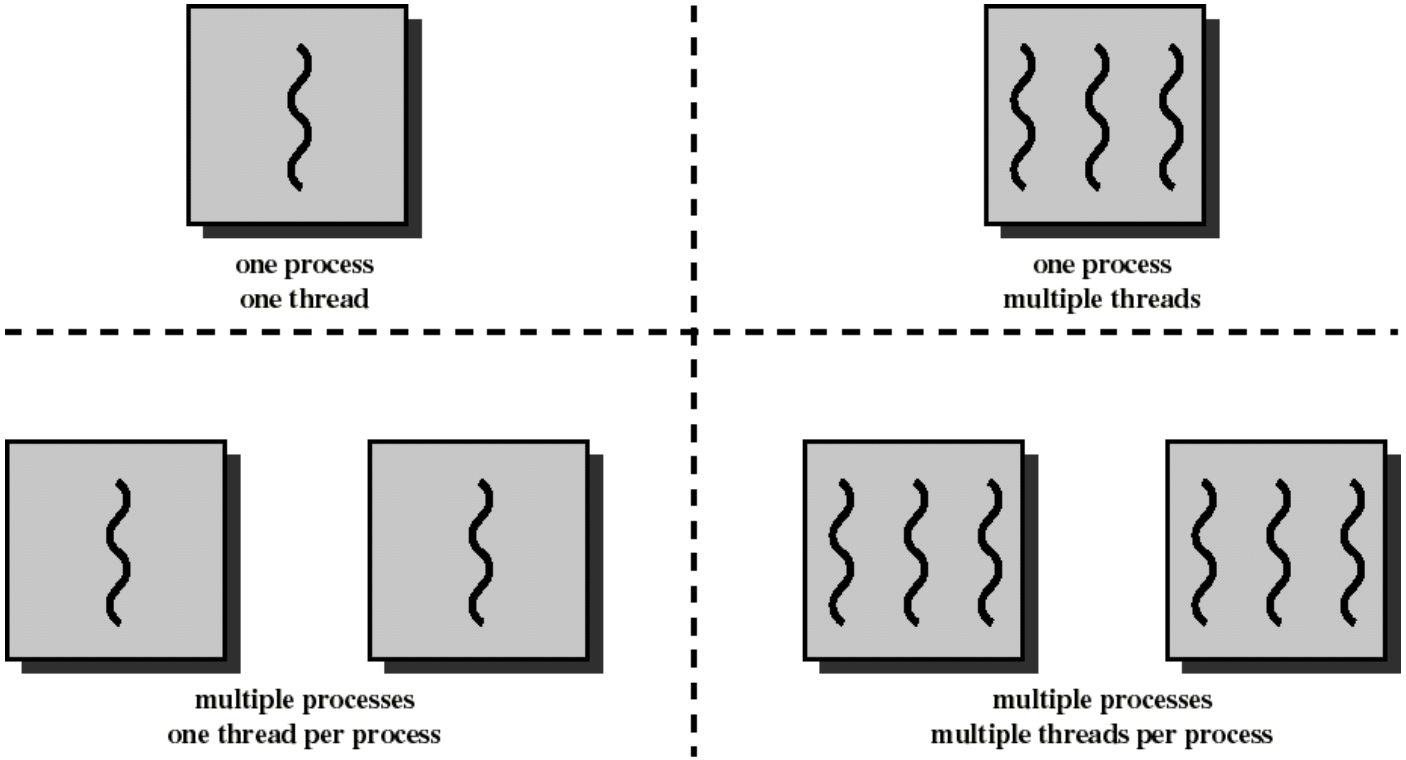
- Bir uygulamada, %75 paralel ve %25 eşzamanlı çalışılıyorsa (S=0,25), **2 çekirdeğe** (N=2) sahip sistemde bu uygulamayı çalıştırınca **1,6 kat hız artar.**
- **Çekirdek sayısı 4** olduğunda, **2.28 kat hız artışı sağlanır.**
- Çekirdek sayısı sonsuza giderken hız artışı (1/S) ‘e doğru gider.
- Intel CPU’lar her çekirdek için 2 thread destekler.

Process ve Thread Kullanımına göre Programlama Modelleri

- Tek Process Programlama Modeli
- Çoklu Process Programlama Modeli
- Tek Process Çoklu Thread Programlama Modeli
- Çoklu Process Çoklu Thread Programlama Modeli

- MS-DOS, işletim sistemi tek bir kullanıcı process'ini ve tek bir thread'i destekler.
- Geleneksel UNIX, çoklu kullanıcı process'lerini destekler, ancak her process için yalnızca bir thread destekler.
- Solaris, Linux ve Windows multi-thread çalışmayı destekler.

Process ve Thread Kullanımına göre Programlama Modelleri



Single Threading **Multi-Threading**

Tek Process Programlama Modeli

- Uygulamanın daha kolay kodlanmasını sağlar.
- Yalnızca tek bir process dizisiyle programın akışının tahmin edilmesi ve hata ayıklaması daha kolaydır.
- Senkronizasyon sorunları yoktur.

- Paralel işlem gücünden tam olarak faydalanamaz.
- Özellikle çok çekirdekli işlemcilerin yaygınlaşmasıyla, bu model performans açısından yetersiz kalabilir.
- Eğer process bloke olursa (örneğin, G/Ç bekleniyorsa), tüm process süreci durur.
- Bu model, basit uygulamalar veya belirli durumlar için uygun olabilirken, daha karmaşık ve yüksek performanslı sistemler için yetersiz kalabilir.

Çoklu Process Programlama Modeli

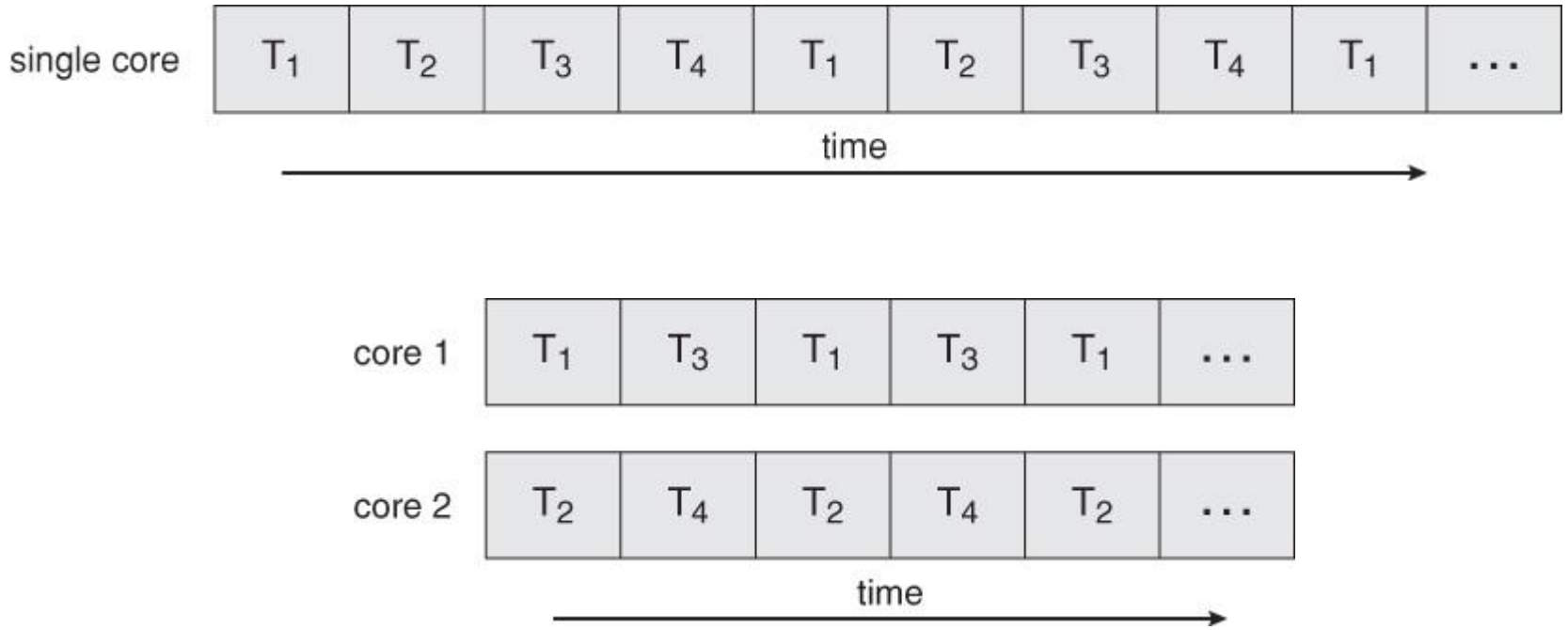
- Çok çekirdekli işlemciler birden fazla process'i gerçek paralel olarak yürütebilir. Her çekirdek, kendi komut setini işleyebilir ve kendi process'ini yürütebilir; bu da paralel işleme ve gelişmiş genel performansa olanak tanır.
- CPU yoğun görevler için etkilidir. Veri analizi ve bilimsel hesaplamalar gibi görevler için ideal olup, süreç izolasyonunu sağlayarak bellek bozulması riskini azaltır.
- Senkronizasyon sorunları ortaya çıkabilir.
- Yazılım geliştirme ve bakımını daha karmaşık hale getirebilir. Farklı process'lerin etkileşimi, hata ayıklama sürecini zorlaştırabilir ve kodun karmaşıklığını artırabilir.

Tek Process Çoklu Thread Programlama Modeli

- Uygulamalar, tek process içerisinde çok sayıda thread'e sahip olacak şekilde geliştirilebilir.
- Bir process içerisinde çoklu thread'ların (multi-threading) tek çekirdekli bir işlemcide yürütülmesi, multi-programming yapısına sahip bir işletim sisteminde birden fazla process'in tek çekirdekli bir işlemcide aynı anda (işlemcide zaman paylaşımı ile) yürütülmesine benzerdir.
- Tek bir process içinde birden fazla thread varsa, birden çok işlemci veya işlemci çekirdeğinde birden fazla thread eşzamanlı yürütülebilir. Bu process'in yürütülmesini daha hızlı yapacaktır.

Tek Process Çoklu Thread Programlama Modeli

- Geleneksel tek çekirdekli işlemci üzerinde bir process'in çoklu thread'ları zaman paylaşımlı olarak eşzamanlı yürütülmek zorundadır (görünürde paralel).
- Bununla birlikte, çok çekirdekli bir işlemci üzerinde thread'lar, gerçek paralel yürütmeye izin verecek şekilde mevcut çekirdeklere dağıtılarak yürütülebilir (gerçek paralel).



Tek Process Çoklu Thread Programlama Modeli

- Çoklu thread, programcılar ve sistem için tasarımda karmaşıklık, senkronizasyon, test etme, hata ayıklama gibi bazı zorluklar getirebilir.

- Multi-thread yazılım geliştirmede thread'ların aynı adres uzayına erişmeleri sonucunda oluşabilecek kilitlenmelerin sezilmesi ve giderilmesi zorunludur. Thread'lar arasında koruma yoktur.

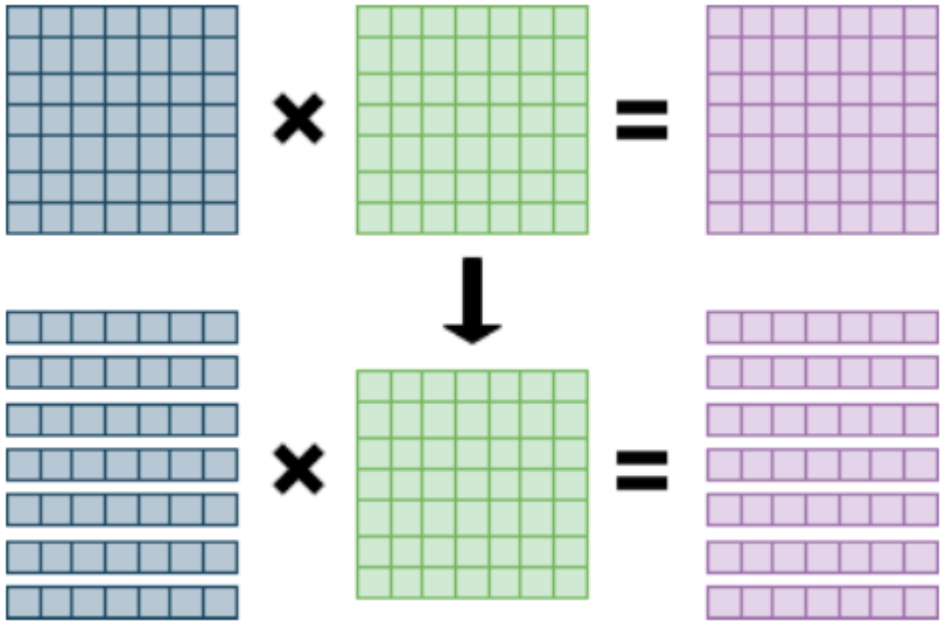
Çoklu Process Çoklu Thread Programlama Modeli

- Çoklu process çoklu thread modelde, kritik görevler ayrı process'ler olarak kodlanır. Her bir process, aynı zamanda çoklu thread'lerden oluşur.
- Çoklu process çoklu thread model daha fazla verim, daha iyi yanıt verme, kaynak verimliliği ve paralellik gibi çeşitli avantajlar sunabilir.
- Örneğin Oracle veritabanı, Chrome ve Firefox web tarayıcıları bu modele göre gerçeklenmişlerdir.

- Bu programlama modeli daha karmaşık bir programlama yaklaşımı gerektirir.

Çoklu thread performans karşılaştırması

- Matris çarpımı : Bu deneyde C = AB ile hesaplanan basit matris çarpımı kullanılmıştır . Matrislerin boyutu 512 × 512'dir. Önce tek bir thread, ardından 2, 4, 8, 16 ve 32 thread boyutuna sahip birkaç senaryo çalıştırılıyor. Kodlar, Java, Go ve Python’da gerçekleştiriliyor.



Çoklu thread performans karşılaştırması

TABLE I
HARDWARE OVERVIEW

Computer model	MacBook Pro (13-inch, 2018)
CPU	2.3 GHz Quad-Core Intel Core i5
RAM	8 GB 2133 MHz LPDDR3
Operating system	MacOS Catalina 10.15.7

TABLE II
SOFTWARE OVERVIEW

Java	OpenJDK 11.0.9 (build 11.0.9+11, mixed mode)
Go	go1.16.4 darwin/amd64
Python	3.9.5

TABLE III
MATRIX MULTIPLICATION TIME CONSUMPTION (IN MILLISECONDS)

Threads	Java	Go	Python
Sequential	316	453	93870
2	226	235	89320
4	177	122	91600
8	151	117	94020
16	138	116	93360
32	155	121	95310

Çoklu thread performans karşılaştırması

- Çoklu thread oluşturmanın ek yükü, bir uygulamanın performansını doğrudan etkiler.
- Belirli bir eşiğe ulaşıldıktan sonra thread oluşturmaya devam etmek mantıklı değildir, çünkü performans düşecektir.

- Bir sorunu çözmek için çoklu thread kullanmak daha iyi bir performans elde edileceğini garanti etmez;
- bazen çoklu thread modeli <u>kullanmamak</u> en iyi seçenek olabilir.

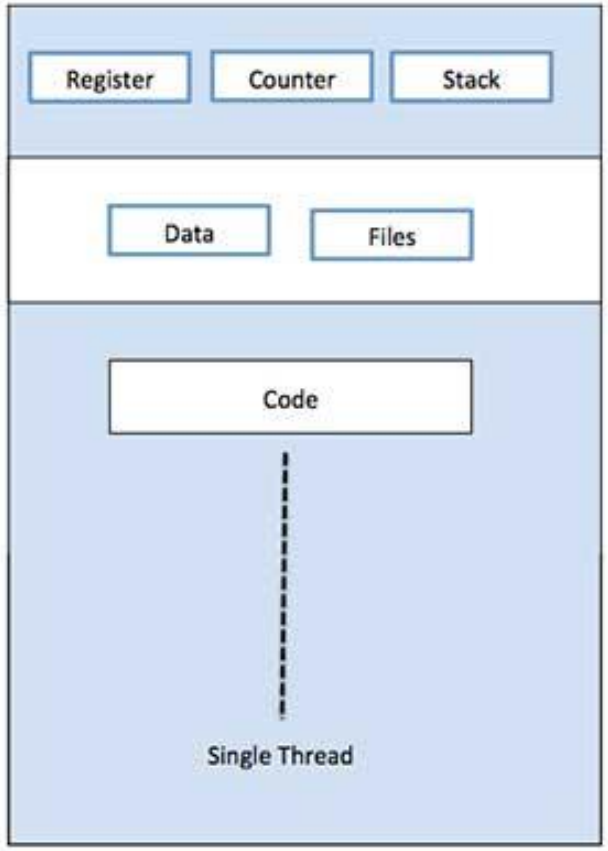
Thread (İşlemcik) Nedir?

- Bir uygulamada, yürütülecek işlerin birbirinden büyük oranda bağımsız olduğu durumlarda çoklu process veya çoklu thread yapısı tercih edilebilir.

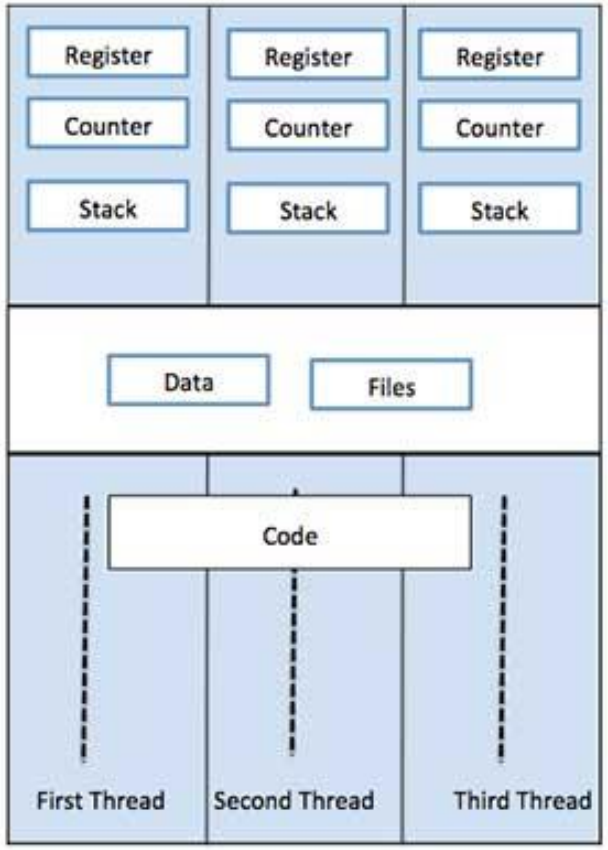
- Uygulamada, yürütülecek işlerin birbirine bağlı olduğu, birlikte yürütüldüğü ve paylaşıldığı durumlarda çoklu thread yapısının kullanılması durumunda bu thread'ların çalışmalarını ortak bir hafızada senkronize etmeleri gerekir.
- İki veya daha fazla thread, paylaşılan verilere eriştiğinde ve doğru senkronizasyon olmadan aynı anda paylaşılan veriyi değiştirmeye çalıştıklarında hatalı bir sonuç üretilmesine sebep olabilirler.

Thread (İşlemcik) Nedir?

- **Single and Multi-thread Processes**
- Multi-thread kullanımında, process'in sahip olduğu tüm thread'ler bu process için ayrılan aynı adres alanını paylaşırlar.
- Thread'lar kendine ait thread ID, program counter, register set ve stack içerir.



Single Process P with single thread



Single Process P with three threads

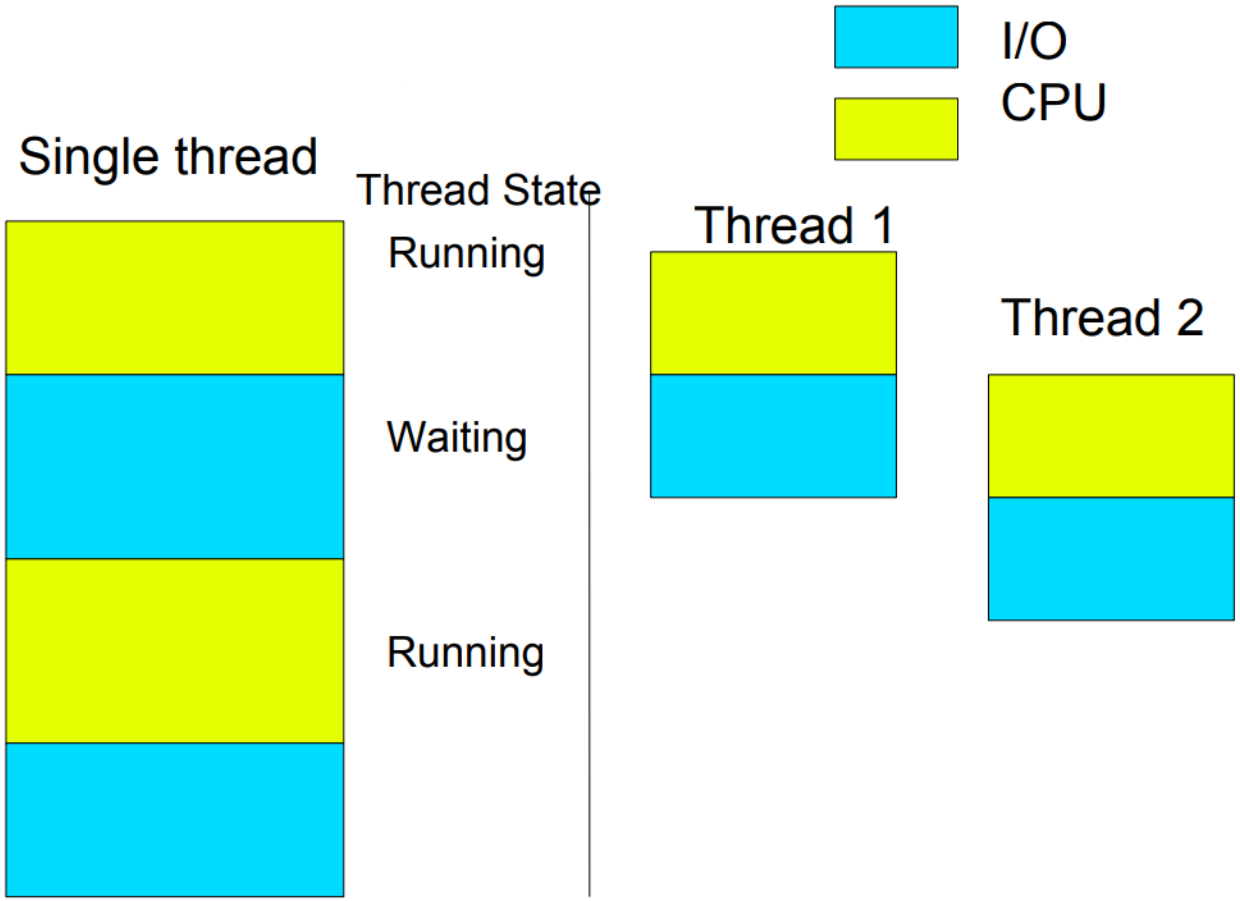
Thread (İşlemcik) Nedir?

- Bir process işlemci çekirdekleri arasında bölünemez, fakat her çekirdeğe ayrı bir thread atanarak işlemci daha verimli kullanılabilir.

- Thread'ların hepsi CPU-bağımlı ise performans kazancı sağlamak çok mümkün olmamaktadır.

- Fakat thread'ların bir kısmı CPU-bağımlı, bir kısmı ise I/O bağımlı ise, bu thread'ların eş zamanlı çalışması, dolayısı ile performans kazancı mümkün olmaktadır.

Thread (İşlemcik) Nedir?

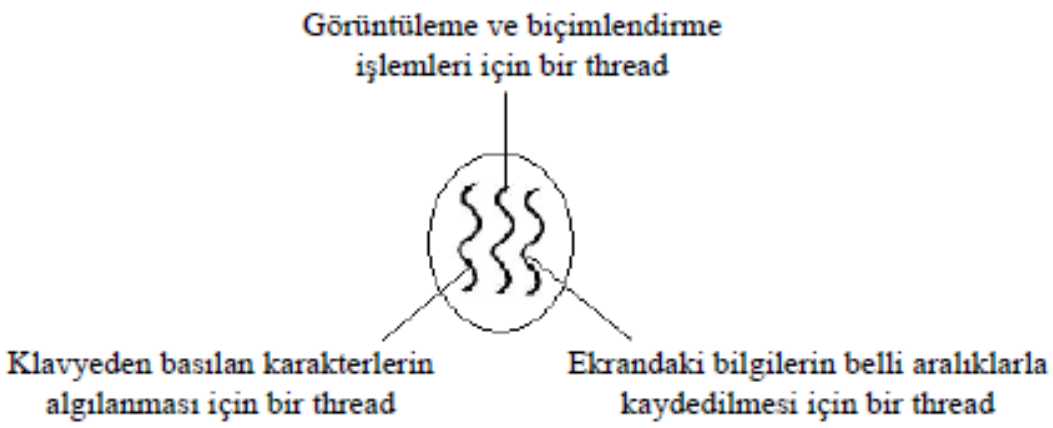


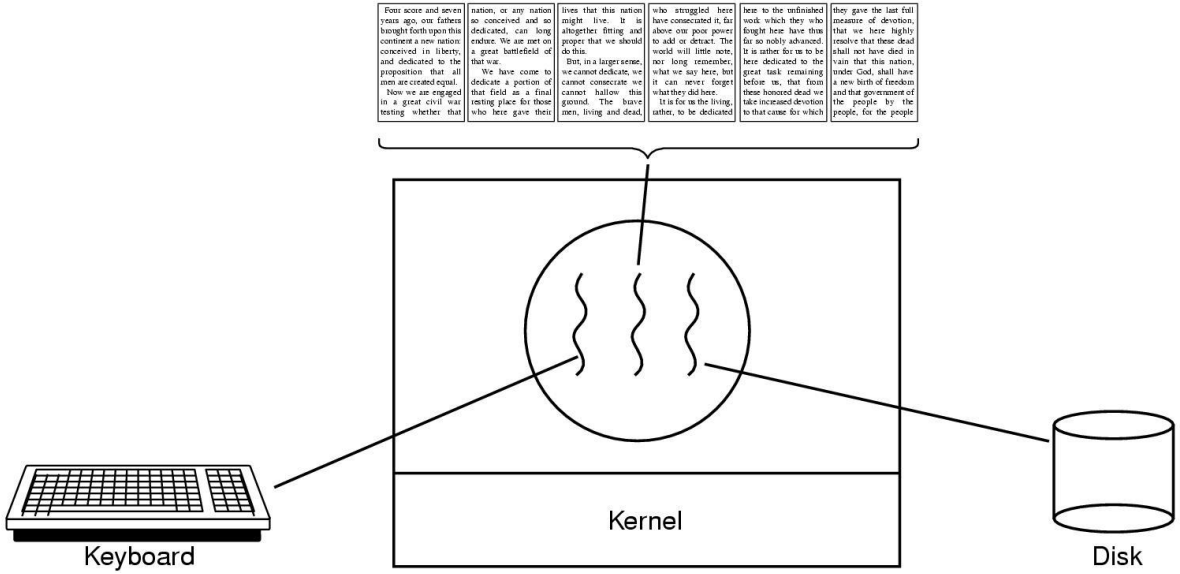
Thread Kullanımı: Chat programı

- Örnek olarak; bir chat programı oluşturulmak istensin.
- Bir chat programı ancak thread mantığı ile yapılabilir.
- İki tane thread metodu oluşturulmalıdır, bir tanesi soketi dinleyen metod, diğeri ise kullanıcıdan giriş bekleyen metod olacaktır.
- Thread'i oluşturma sırasında thread'lardan birine klavyeden giriş alan metodu çalıştırması istenir, diğerinin de soket metodunu çalıştırması sağlanır.

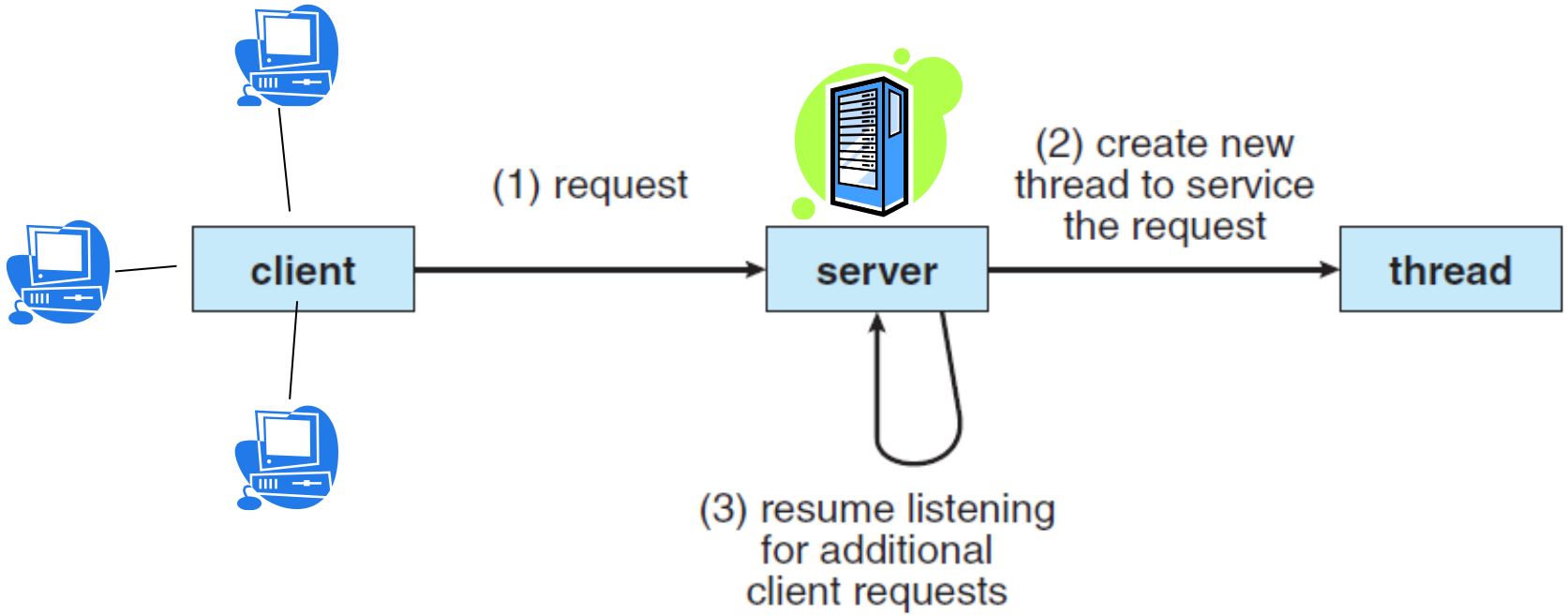
Thread Kullanımı: Kelime İşlemci

- Örneğin, bir kelime işlemci programında, yazma işlemi gerçekleştirilirken, klavyeden bilginin alınması, ekrana basılması ayrı ayrı gerçekleşen iki thread'dır.
- Bunun yanında bir diğer thread yazım ve dilbilgisi hatalarını denetlerken, bir başka thread düzenlenmiş dosyanın otomatik yedeklemelerini yapabilir.
- Bu yüzden thread kullanışlıdır, çünkü bir kelime işlemcide farklı **mantıksal olarak paralel bir şekilde çalıştırılan** görevler bulunur.





Thread Kullanımı: Web Server



Bir web sunucu, her client (istemci) ile ayrı ayrı ilgilenmelidir.

Bir Web sunucu process'i multi-thread çalışırsa, her gelen istek için ayrı bir thread oluşturulur ve process portu dinlemeye devam eder. Birden çok işlemci varsa: farklı thread'lar aynı anda farklı işlemcilerde yürütülebilir.

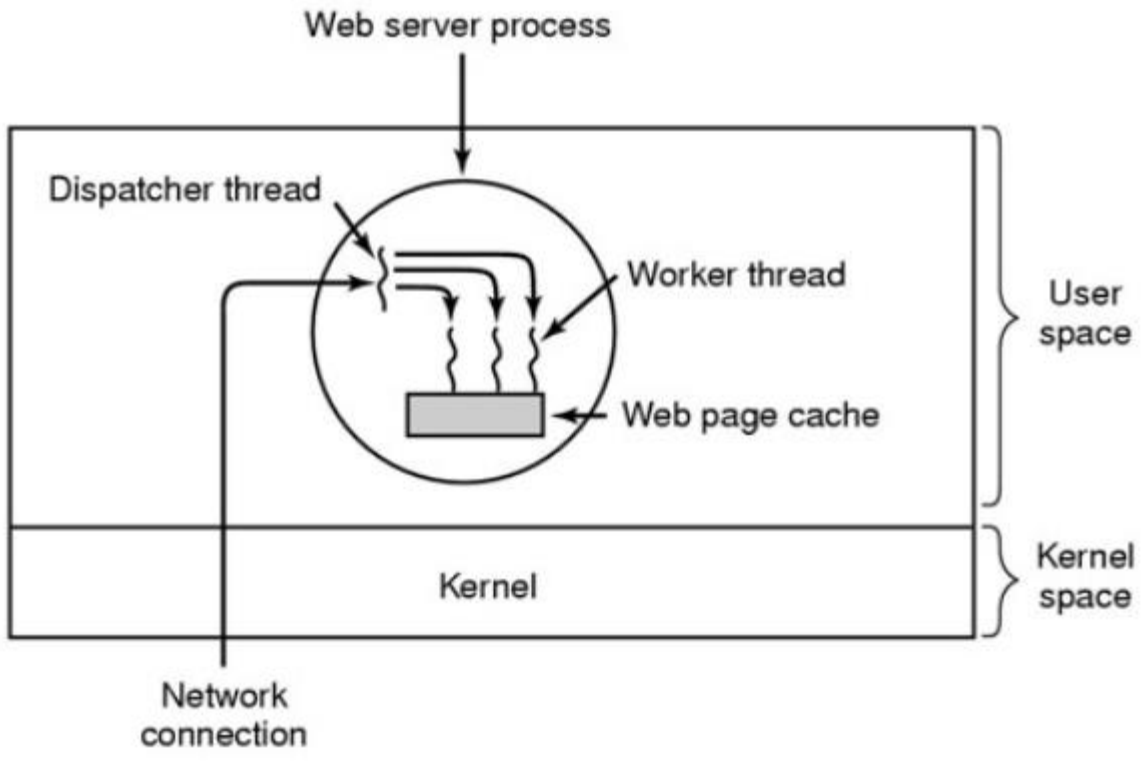
Fakat bu yöntem bazı sorunlar da içermektedir: Thread oluşturma ve sonra iptal etmenin getirdiği yük. Thread sadece o bağlantı için kullanılmış olacak. Sınırsız sayıda thread sonucu web sunucusu çökebilir.

Daha iyi çözüm: sabit bir boyut bir thread havuzu kullanımı.

Thread Kullanımı: Web Server

Bir web sunucuya istemciden çeşitli sayfa istekleri geliyor ve sunucu bu isteklere cevap veriyor.

Bu sunucuda, bir **dispatcher thread** network üzerinden gelen talepleri değerlendirir ve boşta beklemekte olan **worker thread**'lardan birini görevlendirir.



Thread Modeli: Context Switching

- İşletim sistemi için, process oluştururken bellek tahsisi, ve process için context switching (bağlam anahtarlama) gibi işlemler maliyetlidir.
- Fakat, aynı process'e ait thread'ler arasında context switching daha az maliyetlidir.
- Aynı process içindeki thread'ler arasında gerçekleşen context switching zamanı, process'ler arasında gerçekleşen context switching zamanından oldukça küçüktür.

Thread Modeli: Context Switching

- Thread context switching yine de şunları gerektirir.
  - Register set, Program Counter (PC) ve Stack bilgilerini kaydetme veya geri yükleme.
  - Ama bellek yönetimi ile ilgili bir iş yok, dolaysıyla thread context switching kısmen verimlidir.

- ***<u>Farklı</u>*** **processlere ait thread'ler arasında context switching** ***maliyeti yine de fazladır.***

Thread (İşlemcik) kullanımının avantajları

- Thread’lerin sağladığı faydalar 4 kategori halinde ifade edilebilir:

1. Cevap Verme (Responsiveness):

- Kullanıcı etkileşimli uygulamalarda, uygulamanın bir kısmı, uzun süren işlem yürütüyorsa veya kilitlenmişse , kullanıcı ile etkileşim yapan başka bir kısım çalışmasını sürdürür.

- Sistemin cevap verebilirlik özelliği artmış olur.

Thread (İşlemcik) kullanımının avantajları

2. Kaynak Paylaşımı (Resource sharing):

- Thread'ler aynı adres alanını paylaştıkları için, thread'ler arasında veri aktarımı, process’ler arasındaki veri aktarımına göre çok daha hızlıdır.

- Thread'ler arasındaki veri aktarımı genellikle sadece bellek içindeki işaretçilerin (pointers) değiştirilmesiyle yapılırken, process’ler arası veri aktarımı (Inter process communication-IPC) genellikle verilerin kopyalanmasını veya mesajlaşma mekanizmalarını içerir.

Thread (İşlemcik) kullanımının avantajları

3. Maliyet (Economy):

- Bir process oluşturmak, işletim sistemi tarafından bellek alanı ayırmak, sistem kaynaklarını tahsis etmek ve process kontrol bloğu (PCB) oluşturmak gibi işlemler gerektirdiğinden, maliyetli bir işlemdir.
- Thread’ler ait oldukları process’in kaynaklarını paylaştıklarından dolayı bağlam anahtarlama (context switching) daha düşük maliyetle yapılır.
- Thread'ler sadece kendi yığın (stack) alanlarına ve register değerlerine sahip olduklarından, context switching sırasında sadece bu bilgilerin kaydedilmesi ve geri yüklenmesi yeterlidir. Bu da, process’ler arası context switching'e göre çok daha hızlı ve verimli bir işlemdir.

- (Solaris işletim sisteminde, thread oluşturma process oluşturmaya göre 30 kat daha hızlıdır ve thread’lerde context switching 5 kat daha hızlıdır.)

Thread (İşlemcik) kullanımının avantajları

4. Ölçeklenebilirlik (Scalability):

- Çok çekirdekli işlemcilerde, her bir çekirdek bağımsız olarak bir thread'i çalıştırabilir. Bu sayede, aynı process’e ait birden fazla thread, farklı çekirdekler üzerinde gerçek paralel çalışabilir.

- Tek thread'li bir process, aynı anda sadece bir işlemci çekirdeği üzerinde çalışabilir. İşletim sistemi, bu process’i zaman dilimleme (time-slicing) yöntemitle diğer process’ler ile birlikte çalışmasını sağlar, ancak gerçek anlamda paralellik elde edilemez.

ÖZET olarak
Thread kullanım avantajları

- Bir process'in tüm thread'ları process'in text ve data alanlarını paylaşır.
- Process ve içinde bulunan thread aynı adres alanlarında bulunduğundan aynı verilere erişebilirler.
- Bir thread bir dosyayı okuma için açarsa, aynı process'deki diğer thread'larda bu dosyadan okuyabilir.
- Bir thread sonlandırılması (terminate) bir process sonlandırılmasından daha az zaman alır.
- Aynı process içindeki thread'lar arasındaki context switching, process'ler arasındaki context switching işleminden daha az zaman alır.
- Thread'lar çok çekirdekli/işlemcili sistemlerde faydalıdır. Her çekirdek/işlemciden eş zamanlı yararlanmayı sağlar.

Thread (İşlemcik) kullanımının zorlukları

- Çok thread'li programlama aşağıdaki sıkıntıları da beraberinde getirir.

- **Program karmaşıklığını arttırır, Kod sayısı artar, Okunabilirlik ve yönetilebilirlik zorlaşır.**
- **Test etme ve hata ayıklama (Testing and debugging) zorlaşır.**

Thread (İşlemcik) kullanımının zorlukları

- Çok thread’lı programlamadaki 5 önemli zorluk:

1. Görevleri tanımlama (Identifying tasks):

- Çok thread'li bir program tasarlarken, uygulamanın hangi bölümlerinin veya işlemlerinin bağımsız thread'ler halinde çalışabileceğini belirlemek kritik bir adımdır.

- İşlerin ne kadar küçük parçalara ayrılacağına karar vermek önemlidir. Çok küçük parçalara ayırmak, thread yönetiminden kaynaklanan ek yükü artırabilirken, çok büyük parçalara ayırmak paralelleştirme potansiyelini sınırlayabilir.

Thread (İşlemcik) kullanımının zorlukları

2. Denge (Balance):

- Çok thread'li programlamada "Denge (Balance)", iş yükünün thread'ler arasında eşit olarak dağıtılmasını ifade eder ve performans, kaynak kullanımı ve ölçeklenebilirlik açısından önemlidir.
- Ancak, dengelemeyi sağlamak çeşitli zorlukları içerir ve dikkatli bir tasarım ve analiz gerektirir.

Thread (İşlemcik) kullanımının zorlukları

3. Veri bölme (Data splitting):

- "Veri Bölme (Data Splitting)", işlenecek verinin daha küçük parçalara ayrılması ve bu parçaların farklı thread'ler tarafından paralel olarak işlenmesi anlamına gelir.
- Bu teknik, özellikle tek bir thread tarafından işlenemeyecek kadar büyük veri kümeleri üzerinde işlem yapılması gereken durumlarda performansı artırmak için kullanılır.
- Ancak, doğru bölme yöntemini seçmek, veri bağımlılıklarını yönetmek ve senkronizasyonu sağlamak gibi zorlukları içerir.

Thread (İşlemcik) kullanımının zorlukları

4. Veri bağımlılığı (Data dependency):

- Bir thread’ın erişeceği verinin diğer thread’larla bağımlılığının incelenmesi gereklidir.
- Bir thread diğerinin sonuçlarına bağlıysa, doğru sırayla erişim sağlamak için thread’ların senkronize edilmesi gerekir.

- Bir thread'in erişeceği verinin başka thread'ler tarafından üretilmiş veya değiştirilmiş olması durumunda, doğru sonuçlar elde etmek ve hataları önlemek için thread'ler arasında senkronizasyon sağlanması gerekir.

Thread (İşlemcik) kullanımının zorlukları

5. Test etme ve hata ayıklama (Testing and debugging):

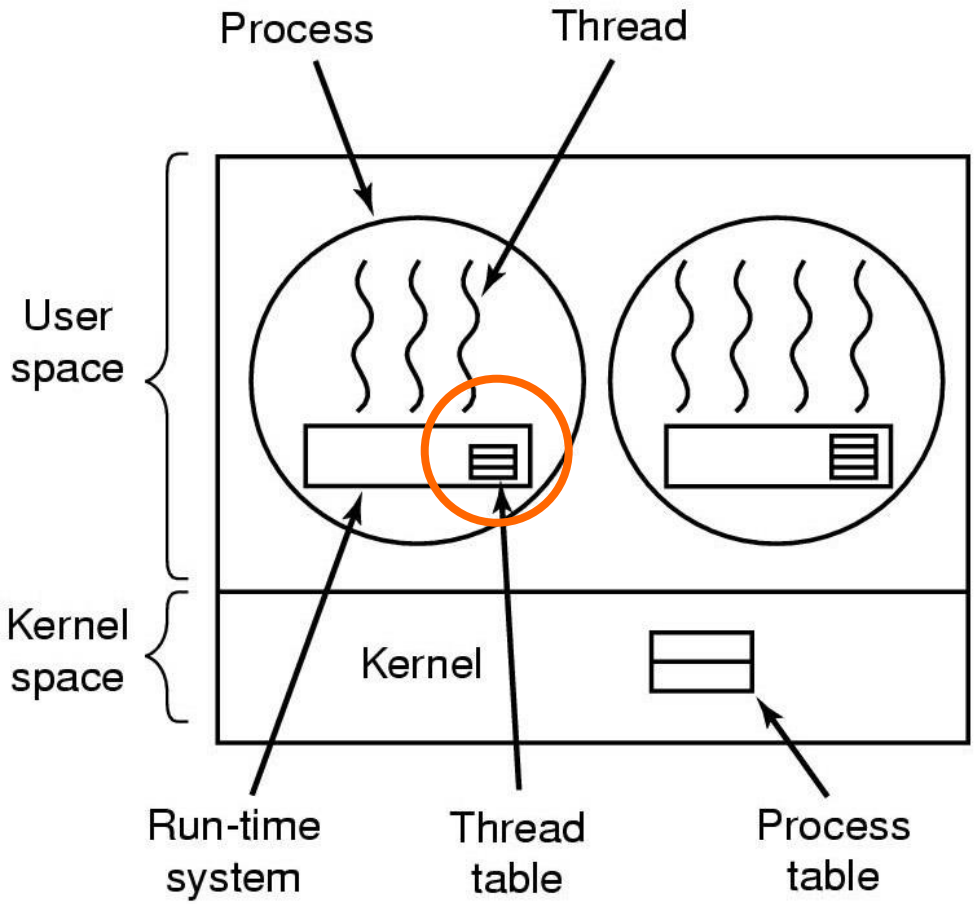
- Çok thread'li programların test edilmesi ve hatalarının ayıklanması, tek thread'li programlara göre çok daha karmaşık ve zordur.

User ve Kernel Threads

- **Thread’lar kullanım açısından;**

- **User thread**
- ve
- **Kernel thread**
- olmak üzere ikiye ayrılır.

User Level Thread



User threads, kernel desteği olmadan yönetilir ve ***Thread tablosu*** *User alanında tutulur.*

Kernel, user thread'larının farkında değildir, sadece process'lerden haberdardır.

User level thread'ler işletim sistemi için görünmez olduğundan, işletim sistemi ile iyi bir entegrasyona sahip değildir.

User level thread

- User thread'ların oluşturulması, zamanlanması ve yönetilmesi kernel'den bağımsız olarak gerçekleştirilir.
- Thread yönetimi user uzayında yapıldığı için her bir process kendi **thread kontrol tablosu**nu tutar. Bu thread tablosunda her bir thread'a ait program counter, stack pointer ve register değerleri tutulur.
- User thread'larının tamamı user adres uzayı içerisinde yürütülür.
- **Bu durumda process içerisindeki thread'lardan kernel habersizdir ve kernel tarafında bu process'e tek thread'dan oluşuyormuş gibi davranılır.**

User level thread

- **Avantajları**
- Thread anahtarlama, kernel modu ayrıcalıkları gerektirmez. Kernel müdahalesi olmadan thread'ler arasında geçiş yapılabilir.
- User level thread herhangi bir işletim sisteminde çalışabilir. İşletim sistemi bağımsızdır.
- User level thread oluşturma ve yönetme daha hızlıdır.
- Yeni bir thread oluşturmak, thread’lar arasında geçiş yapmak ve thread senkronize etmek prosedür çağrısı ile yapılır, yani kernel katılımı yok.
- User level thread’ler için, thread oluşturma, zamanlama ve senkronizasyon gibi işlemler kernel müdahalesi olmadan, user level bir thread kütüphanesi aracılığıyla gerçekleştirir. Bu, kernel sistem çağrıları yapma ihtiyacını ortadan kaldırır ve önemli ölçüde hızlanma sağlar.

User level thread

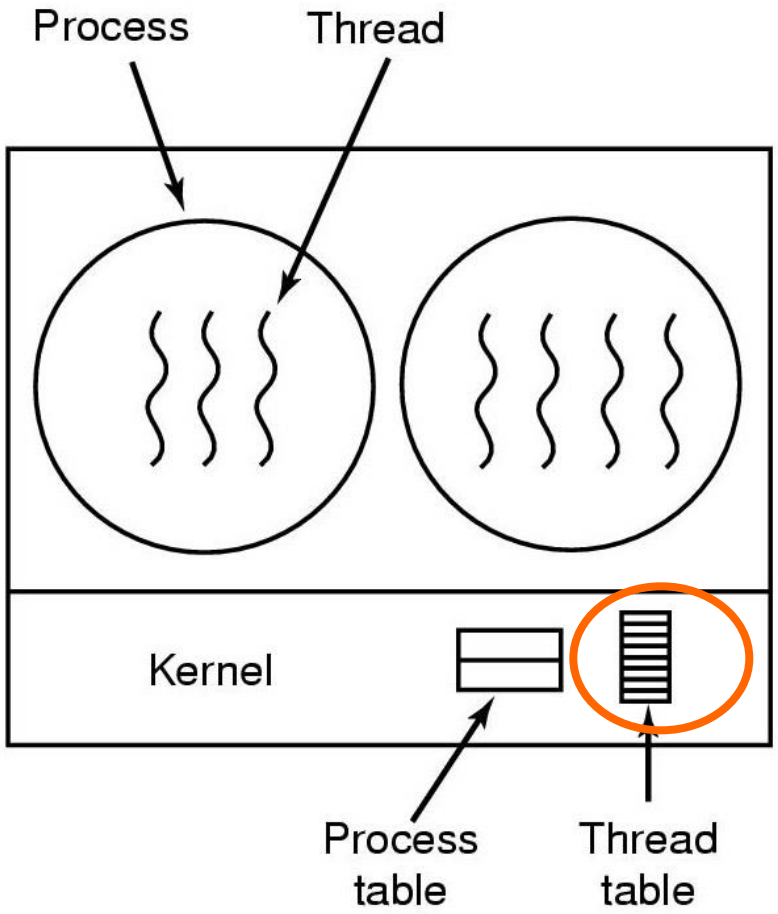
- **Dezavantajları**
- **Bir user seviyesi thread bloklandığında (örneğin, G/Ç işlemi için beklerken), tüm process bloklanır. Çünkü kernel, sadece process’i tanır.**
- **Kernel, thread'leri bilmediği için, farklı thread’leri farklı işlemcilere atayamaz. Bu nedenle gerçek paralellik elde edilemez.**

User level thread

- Bir thread’ın işlemcide yürütülmesine ara verince, işlemci register değerlerinin thread kontrol bloğuna kaydedilmesi, yani thread context switching işini gerçekleştiren prosedürler Run-Time sisteme ait prosedürlerdir.

- Bu prosedürleri çağırmak kernel’ı çağırmaya göre çok daha hızlı bir şekilde gerçekleşir. Bu da thread anahtarlamasının çok hızlı bir şekilde gerçekleştirilmesini sağlar.

Kernel Level Thread



- Kernel threads doğrudan işletim sistemi tarafından desteklenir ve yönetilir.
- Bu durumda, thread oluşturma, zamanlama ve yönetimi Kernel alanında yapılır.
- Kernel, her process'deki ayrı thread'lerden haberdardır.
- Thread tablosu, kernel tarafından yönetilir. Yeni thread oluşturmak için kernel'e sistem çağrısı yapılır.
- Kernel thread'larının oluşturulması ve yönetilmesi user thread'lardan daha yavaştır.

Kernel level thread

- Doğrudan doğruya kernel tarafında oluşturulur.
- Kernel thread’larının oluşturulması, zamanlanması ve yönetimi işletim sisteminin kernel tarafında gerçekleştirilir.
- Kernel, her bir thread'in varlığından haberdardır ve onları ayrı ayrı zamanlayabilir.
- Thread tablosu kernel tarafında tutulur.
- Bir thread yeni bir thread oluşturmak istediğinde ya da yok etmek istediğinde bir sistem çağrısı yaparak kernel’daki thread tablosu üzerinde işlem yapılır.
- Ayrıca kernel, process’ler için bir de process tablosu tutar.

Kernel level thread

- Thread’ların waiting/blocking işlemi yani thread anahtarlaması sistem çağrıları aracılığı ile yapılır.
- Sistem çağrıları Run-Time sistem prosedürlerine göre çok daha maliyetli bir şekilde gerçekleşir.
- Bir kernel thread waiting/blocking konumuna geçtiğinde, sadece o thread bloklanır, diğer thread'ler çalışmaya devam edebilir. Dolaysıyla waiting/blocking işleminden process etkilenmemiş olur.
- Kernel, thread'leri farklı işlemcilere atayarak gerçek paralellik sağlayabilir.

Kernel level thread

- Dezavantajlar:
- Kernel level thread’lar için bağlam anahtarlama (context switching) süresi daha fazladır.

- Kernel thread işlemleri (oluşturma, yok etme ve yönetimi vb.) sistem çağrıları aracılığı ile yapıldığı için fazla sayıda kernel thread işlemi sisteme ağır bir yük getirir.

Thread kütüphaneleri

- Thread kütüphaneleri, programcılara çoklu thread'li uygulamalar geliştirmek için gerekli olan API'leri (Application Programming Interface) sağlayan yazılım kitaplıklarıdır.
- Bu kütüphaneler, thread oluşturma, sonlandırma, senkronizasyon ve zamanlama gibi temel thread işlemlerini kolaylaştırmak için kullanılır.
- Thread kütüphaneleri, temelde iki kategoriye ayrılır:
- **User-Level Thread Libraries**
- **Kernel-Level Thread Libraries**

User-Level Thread Libraries

- Bu kütüphaneler, kernel tarafından değil, user level bir process içinde çalışır.
- Thread oluşturma, zamanlama ve senkronizasyon işlemleri user seviyesinde yapılır.
- User-Level, kütüphanedeki bir fonksiyonun çağrılması, user alanında bir sistem çağrısıyla değil, yerel bir fonksiyon çağrısıyla sonuçlanır.
- Hızlı thread oluşturma ve yönetimi.
- Çekirdek müdahalesi olmadan thread'ler arasında geçiş yapılabilir.
- İşletim sistemi bağımsızdır.
- Dezavantajlar:Bir user level thread bloklandığında, tüm process bloklanır. Kernel, thread'leri bilmediği için, gerçek paralellik elde edilemez.

Kernel-Level Thread Libraries

- Bu kütüphaneler, doğrudan kernel tarafından desteklenir.
- Thread oluşturma, zamanlama ve senkronizasyon işlemleri çekirdek tarafından yapılır.
- Bir kernel level thread bloklandığında, sadece o thread bloklanır, diğer thread'ler çalışmaya devam edebilir.
- Kernel, thread'leri farklı işlemcilere atayarak gerçek paralellik sağlayabilir.
- Dezavantajlar:Thread oluşturma ve yönetimi, user level thread'lerine göre daha yavaştır. Kütüphane için API'de bir fonksiyonun çağrılması genellikle kernele bir sistem çağrısıyla sonuçlanır.
- İşletim sistemi bağımlıdır.

Thread kütüphaneleri

- Üç ana thread kütüphanesi:
- POSIX Pthreads, Win32 Threads ve Java Threads.

- **POSIX Pthreads (Portable Operating System Interface Threads):**
- En yaygın thread kütüphanelerinden biridir. Çoğu Unix benzeri işletim sisteminde (Linux, macOS, FreeBSD vb.) bulunur.
- Hem user level hem de kernel level thread'lerini destekler (işletim sistemine bağlı olarak).
- C dilinde yazılmıştır, ancak diğer dillerde de kullanılabilir.

Thread kütüphaneleri

- **Windows Threads:**
- Windows işletim sistemleri için kullanılan thread kütüphanesidir.
- Kernel level thread'lerini destekler.
- C ve C++ dillerinde kullanılır.
- **Java Threads:**
- Java dilinin yerleşik thread desteğidir.
- Java Virtual Machine (JVM) tarafından yönetilir.
- İşletim sistemi bağımsızdır (JVM'nin çalıştığı herhangi bir platformda çalışır).

Thread kütüphaneleri

- **.NET Threads:**
- .NET Framework ve .NET Core tarafından sağlanan thread kütüphanesidir.
- C#, VB.NET ve diğer .NET dillerinde kullanılır.
- **Python threading Modülü:**
- Python dilinin yerleşik thread desteğidir.
- POSIX Pthreads veya Windows Threads API gibi temel thread kütüphanelerini kullanır.
- Global Interpreter Lock (GIL) nedeniyle gerçek paralellik sınırlı olabilir.

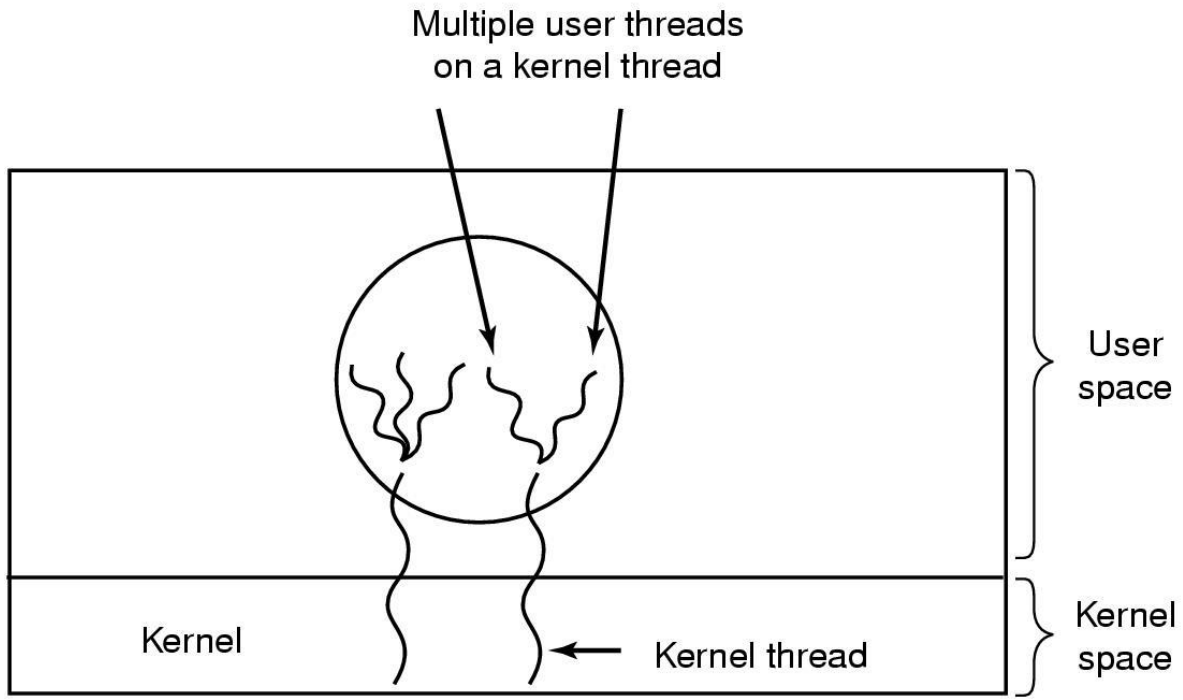
Multi-thread Modelleri

- **Multicore sistemler üzerinde multi-thread bir yürütme için user thread'lar ile kernel thread'lar arasında bir ilişki sağlanmak zorundadır. Bunun için user thread'lar, kernel thread'lar ile eşleştirilir.**
- İşletim sistemlerinin uyguladığı tipik **Multi-thread modelleri** mevcuttur. Bu modeller, user-level threads ile kernel-level threads arasındaki ilişkiyi tanımlar.
- Bu sayede process'ler içindeki birden çok thread, birden çok işlemcide gerçek paralel olarak yürütülebilir.
- Multi-thread modellerin gerçekleştirilmesi için, user thread'lar kernel thread'lara haritalanarak eşleme yapılır.

User thread’lar neden bir kernel thread ile eşlenmelidir?

- User thread’lar, herhangi bir kernel desteği olmaksızın tamamen user alanında yönetilirler.
- İşletim sistemi kerneli, user thread’ların varlığı hakkında hiçbir fikre sahip değildir, çünkü bunlar process’in adres alanında oluşturulmuştur, dolayısıyla kernelin onlar üzerinde kontrolü yoktur.
- Ancak user thread’larını yürütmek için işletim sistemi kernelinin CPU zamanı ve bellek gibi sistem kaynaklarını tahsis etmesi gerekir.
- Bu sistem kaynaklarını düzgün bir şekilde yönetmek ve planlamak için user thread’lar kernel thread’lar ile eşleştirilir.

Multi-thread Modelleri



Kernel sadece kernel level threadlar'dan haberdardır ve sadece onları anahtarlar.

Birden fazla işlemci veya işlemci çekirdeği üzerinde Multi-thread bir yürütme için User level thread'lar, kernel level thread'lar üzerinden işlemciye anahtarlanmaktadır.

user level thread'lar, kernel level thread'lar üzerine yönlendirilir.

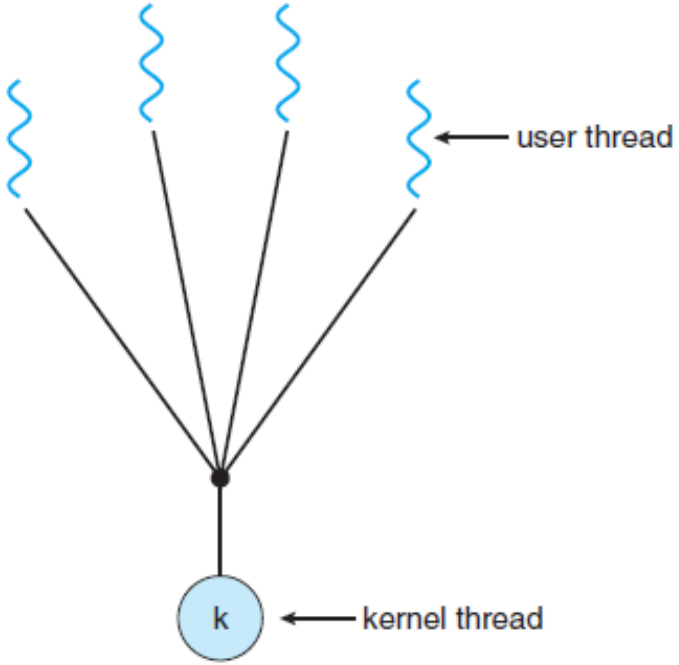
Multi-thread Modelleri

- Kernel ve user thread'lar arasındaki ilişkiye göre, bir process üç multi-thread modelden birini kullanabilir:
- User thread'lar, aşağıdaki stratejilerden birini kullanarak, kernel thread'lar ile eşlenmelidir.

- Çoktan-Bire Modeli (Many-to-one model)
- Bire Bir Modeli (One-to-one model)
- Çoktan Çoğa Modeli (Many-to-many model)

Çoktan-Bire Modeli (Many-to-one model)

- Many-to-one modelinde, çok sayıda user thread'i tek bir kernel thread'i ile eşleştirilir.
- Thread yönetimi thread kütüphanesi vasıtasıyla user alanında yapılır.
- Aynı anda sadece bir tane user thread'i kernel thread'e erişebilir.
- Aynı anda kernel katmanında **sadece bir** thread bulunabildiği için işlemci üzerinde sadece bir thread koşturulabilir.



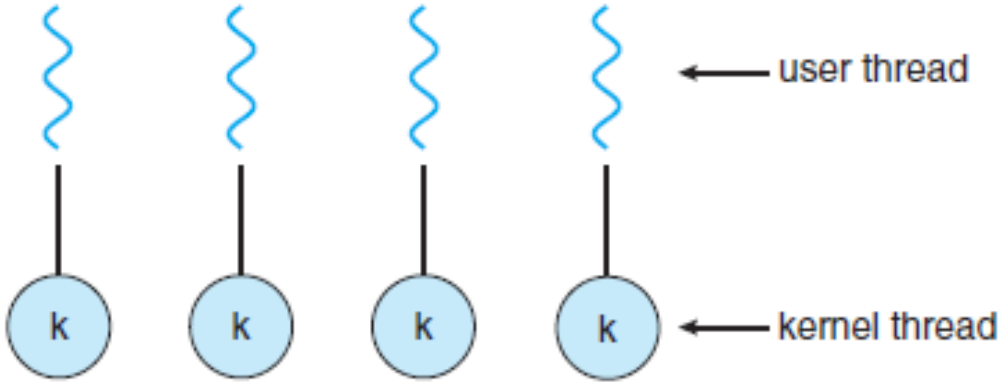
Çoktan-Bire Modeli (Many-to-one model)

- **Sadece bir kernel thread'i kullanıldığı için multicore sistemlerde birden fazla thread için gerçek paralel çalışma yapılamaz.**
- Eğer thread'lerden biri I/O veya herhangi bir şey için (örneğin page-fault) blocked/waiting konumuna geçerse, kernel işlemin sadece bir thread tarafından gerçekleştirildiğini düşündüğünden thread'ler arası geçiş yapamaz, böylece process içerisindeki tüm thread'ler blocked/waiting konumuna geçer.

Bire Bir Modeli (One-to-one model)

- One-to-one modelinde, herbir user thread'i bir kernel thread'i ile eşleştirilir.
- Bu nedenle işlemci üzerinde birden fazla thread multicore sistemlerde gerçek paralel koşturulabilir.
- Her bir user thread için bir kernel thread oluşturulması gereklidir. User thread oluşturulunca kernel thread'de oluşur.
- Many-to-one modeline göre daha fazla eşzamanlılık sağlar (concurrency).

***(Concurrency** (eşzamanlılık) : birden fazla thread arasında kısa aralıklarla geçiş yaparak birlikte ilerletilmesini sağlar.)*

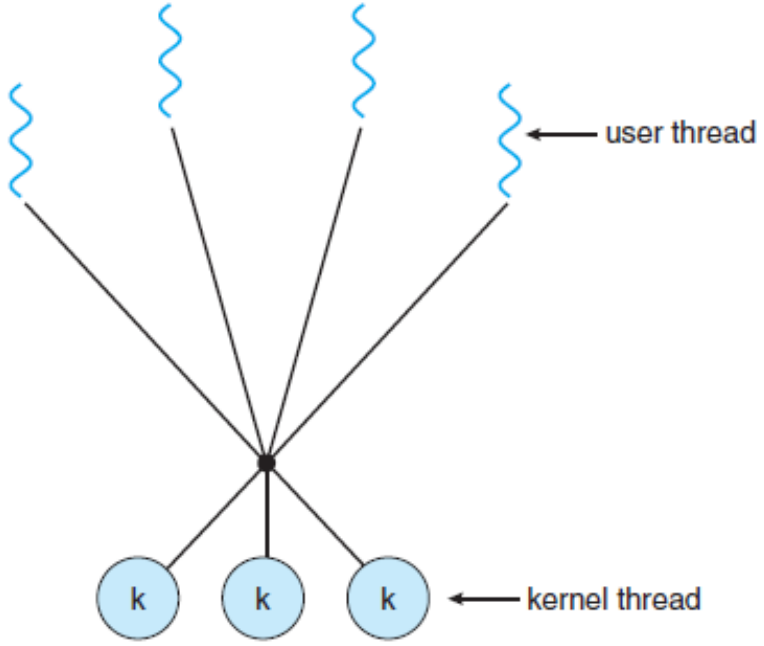


Bire Bir Modeli (One-to-one model)

- Eğer thread'lerden biri G/Ç veya herhangi bir şey için (örneğin page-fault) blocked/waiting konumuna geçerse, process içerisindeki diğer thread'ler çalışmaya devam eder.
- One-to-one model daha iyi bir eş zamanlılık sağlarken, kullanıcı bir uygulamada çok fazla sayıda thread oluşturmamalıdır.
- Kernel thread oluşturmanın getirdiği ek yük performansı bir derece etkiler.
- Bu yüzden process başına thread sayısı bazen kısıtlanır.

Çoktan Çoğa Modeli (Many-to-many model)

- Many-to-many modelinde, **çok sayıda user thread'i ile aynı sayıdaki veya daha az sayıdaki kernel thread'i** eşleştirilir.
- Uygulama geliştiriciye istediği kadar user thread oluşturma şansı verir.
- Kernel threadler multicore sistemler üzerinde gerçek paralel çalışır.



Multi-thread Modelleri

Model	User Thread'leri	Kernel Thread'leri	Bloklanma Durumu	Paralellik	Karmaşıklık
Çoktan Bire	Çok	Bir	Tüm Process Bloklanır	Görünürde Paralellik	Düşük
Bire Bir	Bir	Bir	Sadece İlgili Thread	Gerçek Paralellik	Orta
Çoktan Çoka	Çok	Çok	Sadece İlgili Thread	Gerçek Paralellik	Yüksek